

ARK-450: Verified Action Substitution Attack — PREREGISTRATION

Experiment ID: ARK-450

Title: Verified Action Substitution Attack

Question: Can authorization be detached from the exact governed action?

Type: Classical/Software (no QPU or cryptographic hardware)

Preregistration Date: 2026-07-17

Author: Remnant Fieldworks Inc. — ExecutionProof Research Program

Series: ExecutionProof ARK (Authorization Reality Kernel)

1. Research Question

Can an authorization proof granted for action A be misused to execute a different action B?

In enterprise authorization systems, a critical security property is that approval for one action must not be transferable to any other action — even superficially similar ones. This experiment tests whether ExecutionProof’s binding mechanism strictly couples authorization to the EXACT action parameters, defeating seven classes of substitution attacks:

1. **Destination substitution:** Approve transfer to Alice, execute to Bob
2. **Amount inflation:** Approve \$100 transfer, execute \$10,000
3. **Identity swap:** Approve action for AgentA, execute as AgentB
4. **Tool/resource swap:** Approve file read, execute file write
5. **Partial path matching:** Approve “accounts/alice”, execute “accounts/alice_backup”
6. **Hash-prefix collision:** Craft actions with similar hash prefixes
7. **Metadata manipulation:** Swap contextual fields while preserving core operation

Hypothesis: The authorization guard will ALLOW actions that exactly match the approved action in ALL fields, and DENY any action where ANY field differs (fail-closed against substitution).

2. Experimental Design

2.1 Architecture

- **Decision count:** 800 total (8 arms × 100 scenarios per arm)
- **Dual independent guards:**
 - **V1 (JavaScript):** Zero dependencies, pure logic
 - **V2 (Python):** Independent implementation, same decision procedure
- **Concordance requirement:** V1 and V2 must agree on 100% of decisions
- **Effectiveness gate:** Kill-gate calibration with substitution-effectiveness oracle

2.2 Scenario Structure

Each scenario contains:

```
{
  "scenario_id": "unique identifier",
  "approved_action": {
    "operation": "transfer|read|write|delete|approve|...",
    "principal": "agent identifier",
    "target": "destination/resource path",
    "amount_usd": "numeric value for financial ops",
    "resource_type": "file|account|token|...",
    "metadata": {
      "timestamp": "ISO-8601",
      "context": "additional context",
      "source": "origin identifier"
    }
  },
  "executed_action": {
    // Same structure as approved_action
    // May be identical (baseline) or differ in one or more fields (attack arms)
  },
  "substitution_type": "none|destination|amount|identity|tool|partial_path|
hash_collision|metadata",
  "arm_id": 1-8
}
```

2.3 Decision Procedure (Both Guards)

Inputs:

- `approved_action` : The action that received authorization
- `executed_action` : The action being attempted at execution time

Decision Logic:

1. Extract ALL fields from both actions (operation, principal, target, amount_usd, resource_type, metadata.timestamp, metadata.context, metadata.source)
2. Compare each field:
 - If `approved_action[field] === executed_action[field]` for EVERY field → continue
 - If ANY field differs → immediately return `DENY` (fail-closed)
3. If all fields match exactly → return `ALLOW`

Critical property: Exact equality, not similarity. Even “accounts/alice” ≠ “accounts/alice_backup”.

3. Experimental Arms

Arm 1: Baseline — Exact Match (Expect ALLOW)

- **Design:** `approved_action === executed_action` in ALL fields
- **Expected outcome:** 100% ALLOW
- **Purpose:** Establishes baseline that legitimate, unaltered actions pass

Arm 2: Destination Substitution (Expect DENY)

- **Design:** `approved_action.target ≠ executed_action.target`
- **Example:** Approved “accounts/alice”, executed “accounts/bob”

- **Expected outcome:** 100% DENY
- **Attack:** Redirect authorized action to different recipient

Arm 3: Amount Inflation (Expect DENY)

- **Design:** `approved_action.amount_usd < executed_action.amount_usd`
- **Example:** Approved \$100, executed \$5000
- **Expected outcome:** 100% DENY
- **Attack:** Inflate transaction value beyond approval

Arm 4: Identity Swap (Expect DENY)

- **Design:** `approved_action.principal ≠ executed_action.principal`
- **Example:** Approved for “agent_alpha”, executed as “agent_beta”
- **Expected outcome:** 100% DENY
- **Attack:** Use another agent’s authorization

Arm 5: Tool/Resource Swap (Expect DENY)

- **Design:** `approved_action.operation ≠ executed_action.operation` OR `approved_action.resource_type ≠ executed_action.resource_type`
- **Example:** Approved “read” on “file”, executed “write” on “file”
- **Expected outcome:** 100% DENY
- **Attack:** Change operation type while keeping target

Arm 6: Partial Path Match (Expect DENY)

- **Design:** `executed_action.target` is a substring/extension of `approved_action.target` but not identical
- **Example:** Approved “data/public”, executed “data/public/sensitive”
- **Expected outcome:** 100% DENY
- **Attack:** Exploit path hierarchy to access broader scope

Arm 7: Metadata Manipulation (Expect DENY)

- **Design:** Core fields match but `approved_action.metadata ≠ executed_action.metadata`
- **Example:** Different timestamp, context, or source
- **Expected outcome:** 100% DENY
- **Attack:** Preserve primary action but alter context

Arm 8: Hash-Prefix Collision Attempt (Expect DENY)

- **Design:** Actions crafted to have similar hash prefixes but differ in subtle fields
- **Example:** Target “accounts/alice_primary” vs “accounts/alice_primaryBackup”
- **Expected outcome:** 100% DENY
- **Attack:** Create confusion via similarity

4. Primary Success Criteria (Preregistered Thresholds)

C1. Exact-match allowance ($g_allow_exact_min \geq 0.95$):

- Metric: Minimum ALLOW rate across Arm 1 (baseline-exact-match)
- Threshold: ≥ 0.95 (95% of exact matches must be allowed)
- **PASS if C1 ≥ 0.95**

C2. Substitution rejection ($g_deny_substitution_min \geq 0.95$):

- Metric: Minimum DENY rate across Arms 2-8 (all substitution types)
- Threshold: ≥ 0.95 (95% of substitutions must be denied)
- **PASS if C2 ≥ 0.95**

C3. Separation margin ($g_margin \geq 0.90$):

- Metric: $g_allow_exact_min - (1 - g_deny_substitution_min)$
- Interpretation: Effective gap between allowing exact matches and denying substitutions
- Threshold: ≥ 0.90 (90% separation demonstrates strong binding)
- **PASS if C3 ≥ 0.90**

Overall verdict: PASS if **ALL** of C1, C2, and C3 meet their thresholds.

5. Effectiveness Gate (Kill-Gate)

Purpose: Verify that each substitution arm genuinely contains substitution attempts, not accidental exact matches.

Oracle Logic (per scenario):

```
def is_substitution_effective(scenario):
    approved = scenario['approved_action']
    executed = scenario['executed_action']
    sub_type = scenario['substitution_type']

    if sub_type == 'none':
        # Baseline arm: must be exact match
        return approved == executed
    else:
        # Attack arms: at least one field must differ
        return approved != executed
```

Gate Threshold: 100% effectiveness required per arm. If any arm's scenarios fail the oracle, ABORT before execution (corrupt test design).

Sample size: ~10-15 scenarios per arm for kill-gate calibration.

6. Dual-Guard Concordance

Requirement: V1 (JavaScript) and V2 (Python) must produce identical decisions for all 800 scenarios.

Concordance metric:

```
concordance_rate = (# scenarios where V1 == V2) / 800
```

Threshold: 100% (800/800) required. Any discordance indicates implementation error; result is INVALID.

7. Random Seed & Reproducibility

Generator seed: 450_substitution_attack_2026

Scenario generation: Deterministic based on seed

Reproducibility: Exact scenario set can be regenerated with same seed

Locked artifacts (preregistration):

1. PREREGISTRATION.md (this document)
2. schemas/substitution_scenario_schema.json
3. generator/scenario_generator.py
4. verifiers/v1_guard.js
5. verifiers/v2_guard.py
6. run_killgate.py
7. run_arms.py

Manifest: SHA-256 hashes of all locked files committed BEFORE any execution.

8. Data Collection

Per-arm metrics:

- rate_allow : Fraction of ALLOW decisions
- rate_deny : Fraction of DENY decisions
- v1_v2_concordance : Agreement rate between guards
- substitution_effectiveness : Fraction passing oracle

Overall metrics:

- g_allow_exact_min : Minimum ALLOW rate for Arm 1
- g_deny_substitution_min : Minimum DENY rate across Arms 2-8
- g_margin : Separation metric (C3)
- total_concordance : V1-V2 agreement across all 800 scenarios

Output format: JSON files in results/ directory, human-readable tables in RESULTS.md .

9. Analysis Plan

Primary analysis:

1. Compute per-arm metrics (ALLOW rate, DENY rate)
2. Identify g_allow_exact_min (Arm 1 baseline)
3. Identify g_deny_substitution_min (minimum across Arms 2-8)
4. Compute g_margin separation
5. Evaluate C1, C2, C3 against thresholds
6. Determine overall verdict (PASS/FAIL)

Secondary analysis:

- Which substitution types are most/least effectively blocked?
- Are there any substitution scenarios that leaked ALLOW?

- Concordance analysis: Any V1-V2 discrepancies?

Reporting:

- Full per-arm breakdown
- Overall metrics table
- Verdict with evidence
- Honest reporting of any anomalies or failures

10. Scope & Limitations

In scope:

- Structural substitution attacks (swapping action fields)
- Seven distinct substitution vectors
- Exact equality enforcement
- Dual independent verification

Out of scope:

- Timing-based attacks (ARK-451 covers revocation timing)
- Cryptographic hash collisions (beyond prefix similarity)
- Quantum or hardware-based attacks (classical experiment)
- Dependency degradation (ARK-456 covers fail-closed under dependency loss)

Known limitations:

- Synthetic scenarios, not real-world transaction traces
- Limited to 8 substitution patterns (representative, not exhaustive)
- No adversarial ML-based substitution generation

11. Ethical & Safety Considerations

No real systems affected: All scenarios are synthetic. No actual financial transactions, file operations, or agent executions occur.

Honest reporting: Any failure will be reported transparently. The goal is scientific truth, not marketing.

Fail-closed principle: The experiment tests whether the system defaults to DENY when in doubt — a critical safety property.

12. Timeline & Execution Protocol

1. **Preregistration (LOCK commit):** Commit this document + all code + MANIFEST hashes BEFORE execution
2. **Kill-gate calibration:** Generate small sample, verify effectiveness oracle
3. **Full execution:** Generate 800 scenarios, run both guards, collect results
4. **Analysis:** Compute metrics, determine verdict
5. **Results documentation:** Write RESULTS.md

6. **Publication:** Create GitHub PR (not merged), stage Zenodo draft (not published), await Derek's review

No post-hoc changes: Thresholds, criteria, and decision logic are LOCKED at preregistration. Results stand as executed.

13. Expected Outcome

Hypothesis: The authorization guard will demonstrate perfect action binding:

- $C1 \geq 0.95$: Exact matches consistently allowed (likely ~ 1.00)
- $C2 \geq 0.95$: All substitution attempts consistently denied (likely ~ 1.00)
- $C3 \geq 0.90$: Strong separation between exact and substituted (likely $\sim 0.95-1.00$)

Null hypothesis (failure mode): If ANY criterion fails, it indicates the guard permits authorization detachment — a critical security flaw.

Real-world implication: PASS confirms that ExecutionProof enforces “authorization for action A cannot be used for action B” — defeating credential stuffing, privilege escalation, and scope creep attacks in AI agent deployments.

Preregistration complete. LOCK commit follows. No modifications to design, thresholds, or decision logic after this point.

Document hash: (computed at LOCK commit)

Repository: github.com/derekhone/executionproof-testbeds

Branch: ark-450-substitution-attack

Contact: Remnant Fieldworks Inc. — ExecutionProof Research Program